INTEGRATED PROCUREMENT INTELLIGENCE WORKFLOW

Advanced LangGraph • Four-Agent Orchestration • COD

ProcureFlow AI — Invoice Approval and Procurement Follow-up Workflow

PROBLEM STATEMENT

Northstar Industrial Services receives vendor invoices in inconsistent text layouts. Before an invoice can be paid, the accounts-payable team must extract its identifying details, verify the purchase order, prevent duplicate payment, and apply the company's approval rules. Invoices that cannot be approved require a courteous vendor notification. The reasons for many exceptions are discussed in procurement review meetings, but follow-up actions remain buried inside meeting minutes. Corrective work is consequently left without a clear owner or urgency level.

Build a terminal application that processes one invoice per run. A language model extracts structured invoice data. Python code and LangChain tools perform all payment eligibility decisions. If an invoice is not approved, the workflow finds the procurement meeting associated with its purchase order, extracts follow-up actions, assigns each action to one meeting attendee with a priority, and drafts a vendor note. Approved invoices follow a separate payment branch.

OBJECTIVE

- Extract invoice_number, vendor_name, po_number, amount, and currency from free-form invoice text in exactly one Gemini call, validated against a Pydantic schema.
- Look up the purchase order and check the payment ledger through LangChain tools.
- Apply deterministic checks in a fixed order. The language model must never decide whether an invoice is approved, under review, or rejected.
- Schedule payment only when the invoice passes every check, returning a deterministic payment reference.
- For review or rejected invoices, locate the linked procurement meeting, extract actions from its prose, and assign an owner and priority to each action.
- Draft a structured, polite vendor note for every review or rejected invoice.
- Route approved and non-approved invoices through different graph branches, then format a single final report.
- Return a not-found message without building or invoking the graph when the requested invoice file does not exist.

Agent count The solution must contain exactly four LLM agent functions: extract_invoice_data, extract_follow_up_actions, assign_action_owner_priority, and draft_vendor_note. Tools and deterministic Python functions are not LLM agents.

BUSINESS RULES AND DECISION ORDER

The first failed check determines the invoice decision. Stop checking as soon as a rule decides the outcome.

Order	Condition	Decision	Required reason
1	The quoted PO is missing from the register, or its status is not Open (case-insensitive status comparison is allowed).	rejected	State whether the PO was not found or was not open.
2	The invoice number already exists in data/payment_ledger.csv.	rejected	Identify the duplicate-payment risk.
3	Extracted vendor name differs from the PO vendor name, ignoring surrounding whitespace and letter case.	review	State that the vendor names do not match.
4	Absolute amount difference is greater than 2% of approved_amount.	review	State that the amount is outside the permitted tolerance.
5	None of the checks above fail.	approved	State that all checks passed.

Boundary condition An amount difference of exactly 2% is allowed. Trigger review only when abs(invoice_amount - approved_amount) > 0.02 * approved_amount.

PROJECT STRUCTURE

```
Project/
├── main.py
├── state.py
├── agents.py
├── tools.py
├── nodes.py
├── graph.py
├── tests.py
├── requirements.txt
├── .env
├── .env.example
└── data/
    ├── invoices/
    │   ├── INV-25041.txt
    │   ├── ...
    │   └── INV-25048.txt
    ├── purchase_orders.csv
    ├── payment_ledger.csv
    └── procurement_meetings.json
```

PURPOSE OF FILES

File	Purpose
state.py	Declares the typed state schema carried through the graph.
agents.py	Contains the four LLM agents and their Pydantic reply schemas.
tools.py	Declares deterministic tools for PO lookup, duplicate-payment lookup, and payment scheduling.
nodes.py	Implements graph nodes. Each node returns only the state fields it writes.
graph.py	Builds the conditional LangGraph workflow and returns the compiled graph.
main.py	Validates the requested invoice file, invokes the workflow, and prints the final report.
tests.py	Tests schema contracts, decision precedence, routing, tool calls, agent calls, and edge cases.
data/invoices/	Eight plain-text invoice files, each named by invoice number.
data/purchase_orders.csv	Read-only purchase-order register.
data/payment_ledger.csv	Read-only list of invoices already paid.
data/procurement_meetings.json	Read-only meeting records whose prose contains follow-up actions.
.env / .env.example	Configuration for GEMINI_API_KEY and GEMINI_MODEL; do not hard-code secrets.
requirements.txt	Lists Python dependencies.

DATASET DESCRIPTION

All files under data/ are input fixtures and must never be modified by the application or tools. The package should contain eight invoice files, a purchase-order register with at least 20 rows, a payment ledger, and six procurement meeting records. The fixtures must cover every decision branch and at least one edge case for each rule.

1. Invoice files — data/invoices/

- Each file is UTF-8 plain text and is named INV-25041.txt through INV-25048.txt.
- Invoices use varied formats: email text, tax-invoice layout, compact summary, and narrative letter.
- Invoice number, vendor, PO number, amount, and currency may appear in different positions. Currency may be represented as INR, Rs., or ₹.
- Include realistic distractor values such as dates, quantities, unit rates, tax values, and line amounts. Extract the total payable, not a distractor.
- Across the files, cover approved, review, rejected, duplicate, closed/missing PO, vendor mismatch, and amount-tolerance cases.

2. Purchase-order register — data/purchase_orders.csv

Column	Returned type	Meaning
po_number	str	Unique purchase-order identifier.

vendor_name	str	Vendor recorded against the order.
approved_amount	float	Approved order amount in INR.
currency	str	Currency code; the assessment fixtures use INR.
status	str	Order status such as Open, Closed, or On Hold.
department	str	Department that raised the order.
order_date	str	Date as recorded in the CSV.

The lookup tool returns all seven columns under their CSV names. Convert approved_amount to float and preserve every other value as a string. An unknown PO returns {}.

3. Payment ledger — data/payment_ledger.csv

Required columns: invoice_number and payment_reference. The duplicate-payment tool checks invoice_number using an exact match after trimming surrounding whitespace from the requested invoice number. Return True if it exists and False otherwise. The ledger is read-only.

4. Procurement meetings — data/procurement_meetings.json

The JSON file contains one top-level key named meetings. Each record contains:

Field	Type	Requirement
meeting_id	str	Unique identifier.
po_number	str	Links the meeting to an invoice's quoted PO.
title	str	Meeting title.
attendees	list[str]	Exactly four attendee names per record.
notes	str	One prose block of approximately 800–1,100 characters, with actions embedded in natural discussion.

Match meetings by po_number. Each PO has at most one linked meeting. The notes must not contain a pre-structured action list; the extraction agent must infer follow-up actions from the prose. All four attendees should be mentioned naturally in the notes.

MODULE SPECIFICATION

MODULE 1 — state.py

Declare a typing.TypedDict named ProcurementState with exactly these keys and types:

Key	Type	Writer / purpose
invoice_path	str	Input path supplied by process_invoice.
invoice_text	str	Whole invoice text read by load_invoice.
invoice	dict	Validated invoice fields written by extract_invoice.
purchase_order	dict	PO lookup result or {}.
decision	str	One of approved, review, rejected.
reason	str	Explanation for the first deciding rule.
payment_reference	str	Written only by approve_payment.
meeting_id	str	Linked meeting ID, or empty string.
meeting_title	str	Linked title, or empty string.
attendees	list[str]	Attendees from linked meeting.
notes	str	Meeting minutes, or empty string.
action_items	list[dict]	Each item has task, owner, and priority.
vendor_note	dict	Structured subject and message.
report	str	Final formatted report.

- A graph invocation begins with invoice_path only. Other values are written by their designated nodes.
- Do not add convenience keys to ProcurementState. Use only the specified keys.
- Nodes return dictionaries containing only the fields they changed; do not reconstruct and return the full state.

MODULE 2 — tools.py

Declare the following three functions with the LangChain @tool decorator and a concise one-line description. Call every tool through its invoke method with a dictionary of arguments. No tool calls Gemini.

Tool A: lookup_purchase_order

```
lookup_purchase_order(po_number: str) -> dict
```

- Read data/purchase_orders.csv and find the row whose po_number equals the argument exactly after stripping the argument's surrounding whitespace.
- Return all seven columns. Convert approved_amount to float and preserve every other column as a string.
- Return {} if no row matches; do not raise merely because a PO is absent.

Tool B: invoice_already_paid

```
invoice_already_paid(invoice_number: str) -> bool
```

- Read data/payment_ledger.csv and check for an exact invoice_number match after trimming surrounding whitespace.
- Return True when a row exists and False otherwise. Do not call Gemini or modify the ledger.

Tool C: schedule_payment

```
schedule_payment(invoice_number: str, amount: float) -> str
```

- Return a deterministic reference in the format PAY-{invoice_number}-{amount_rounded_to_nearest_whole_number}.
- Example: INV-25041 with amount 48500.40 returns PAY-INV-25041-48500. Use Python's standard rounding behavior consistently and test the required example.
- This tool simulates payment: it reads and writes no file and makes no network call. Call it only on the approved branch.

MODULE 3 — agents.py

Load GEMINI_API_KEY and GEMINI_MODEL from .env when the module is imported. Create one Gemini client and reuse it. Do not hard-code the API key or model name. Validate model responses against the specified Pydantic schemas where applicable.

Agent 1: extract_invoice_data

```
extract_invoice_data(invoice_text: str) -> dict
```

Pydantic schema: InvoiceData

Field	Type	Constraint
invoice_number	str	Required; preserve identifier as written.
vendor_name	str	Required; extract the vendor, not the billing recipient.
po_number	str	Required; preserve PO identifier.
amount	float	Total payable as a number without currency symbols or digit grouping.
currency	str	Normalize Rs., ₹, and INR representations to INR.

- Make exactly one client.models.generate_content call per invocation, using MODEL and requesting JSON conforming to InvoiceData.
- Extract the total payable, not an individual line amount, unit rate, tax component, date, or PO amount.
- Validate the reply and return a plain dictionary with the five fields.
- If validation fails, allow an error to surface; do not fabricate missing values or silently default them.

Agent 2: extract_follow_up_actions

```
extract_follow_up_actions(notes: str) -> list[str]
```

- Ask the model to identify every actionable follow-up arising from the procurement meeting notes.
- Require one action per output line, each beginning with ACTION: followed by the task text.
- Retain only lines beginning with ACTION: and remove that marker plus surrounding whitespace.
- Return an empty list when no valid ACTION: lines are present.
- Do not add, discard, rewrite, deduplicate, or infer actions in Python; the agent's output determines the action list.

Agent 3: assign_action_owner_priority

```
assign_action_owner_priority(task: str, attendees: list[str]) -> tuple[str, str]
```

- Send one task and the complete attendee list to the model in one instruction.
- Require exactly two output lines: OWNER: <name> and PRIORITY: <High|Medium|Low>.
- The owner must be exactly one supplied attendee name, copied with identical spelling and punctuation.
- Priority must be exactly High, Medium, or Low.
- Parse only the required labelled lines. Do not match owner names by substring, infer priority with keyword rules, or invent a fallback for invalid output.

Agent 4: draft_vendor_note

```
draft_vendor_note(invoice: dict, decision: str, reason: str) -> dict
```

Pydantic schema: VendorNote

Field	Type	Constraint
subject	str	Concise, professional subject line.
message	str	Brief, courteous message stating decision and reason.

- Make exactly one Gemini call and validate the response against VendorNote.
- The prompt must include vendor name, invoice number, PO number, decision, and reason.
- For review, ask the vendor to clarify or correct the discrepancy. For rejection, explain the blocking issue without claiming payment has been scheduled.
- Return the validated subject and message unchanged. Do not call this agent for approved invoices.

MODULE 4 — nodes.py

Implement the following nodes. Each receives ProcurementState and returns only the fields it writes.

load_invoice(state: ProcurementState) -> dict

- Read the entire file at invoice_path as UTF-8 text.
- Return invoice_text. Do not extract fields in this node.

extract_invoice(state: ProcurementState) -> dict

- Call extract_invoice_data exactly once with invoice_text.
- Store the returned dictionary unchanged under invoice.

match_purchase_order(state: ProcurementState) -> dict

- Call lookup_purchase_order via invoke using invoice[po_number]; store the row under purchase_order.
- Apply the five business checks in the exact order specified above and stop at the first failing check.
- Call invoice_already_paid via invoke using invoice[invoice_number] for duplicate detection.
- Make no Gemini call. Return only purchase_order, decision, and reason.

approve_payment(state: ProcurementState) -> dict

- Reachable only when decision is approved.
- Call schedule_payment via invoke using invoice_number and amount from invoice.
- Store the returned value unchanged under payment_reference. Do not call Gemini.

load_review_meeting(state: ProcurementState) -> dict

- Reachable only for review or rejected invoices.
- Read data/procurement_meetings.json and find the record whose po_number matches invoice[po_number].
- Copy meeting_id, title, attendees, and notes unchanged into their state fields. If none exists, use empty strings, an empty attendee list, and empty notes.
- Make no Gemini call.

extract_actions(state: ProcurementState) -> dict

- If notes is empty, make no model call and return an empty action_items list.
- Otherwise call extract_follow_up_actions exactly once.
- Create one action dictionary per task with task set to the task text and owner/priority set to empty strings.
- Preserve order and task text.

assign_owners(state: ProcurementState) -> dict

- Call assign_action_owner_priority once per action item with that task and the full attendees list.
- Preserve task text and order; store returned owner and priority for each item.
- Do not batch tasks or reuse one result across tasks. Make no calls if there are no actions.

notify_vendor(state: ProcurementState) -> dict

- Call draft_vendor_note exactly once with invoice, decision, and reason.
- Store the returned dictionary unchanged under vendor_note.
- This node is not used for approved invoices.

format_report(state: ProcurementState) -> dict

- Always include invoice number, vendor, PO number, decision, and reason.

- For approved invoices, include the payment reference and omit vendor-note and meeting-action sections.
- For review/rejected invoices, include the vendor note and linked meeting details; list every action in order with position, priority, task, and owner.
- If no meeting is linked, explicitly state that no linked procurement meeting was found and no actions were identified.
- Store the finished report under report.

MODULE 5 — graph.py

Router: route_after_matching

```
route_after_matching(state: ProcurementState) -> str
```

- Return approve_payment when decision is approved.
- Return load_review_meeting when decision is review or rejected.
- Use decision only; do not inspect invoice text, reason, or other state fields to choose a branch.

Builder: build_graph

```
build_graph()
```

Create a StateGraph over ProcurementState and register exactly these nine nodes:

- load_invoice, extract_invoice, match_purchase_order, approve_payment, load_review_meeting, extract_actions, assign_owners, notify_vendor, format_report.

Connect the graph with this topology:

```
START -> load_invoice -> extract_invoice -> match_purchase_order
                                    |
                          route_after_matching
                           /              \
                    approved          review/rejected
                        |                    |
               approve_payment       load_review_meeting
                        |                    |
                        |                extract_actions
                        |                    |
                        |                assign_owners
                        |                    |
                        |                notify_vendor
                        |                    |
                        └────────────┐       |
                                     |       |
                                     v       v
                                format_report -> END
```

- Use one conditional branch after match_purchase_order.
- Both branches converge at format_report and then END.
- Return the compiled graph.
- Building the graph must not read data files, invoke an agent, or schedule payment.
- Do not add a second decision-making LLM or duplicate an agent under a different name.

MODULE 6 — main.py

Function: process_invoice

```
process_invoice(invoice_number: str) -> dict
```

- Strip surrounding whitespace from invoice_number.
- Resolve the path as data/invoices/{invoice_number}.txt.
- If the file does not exist, return {} without calling build_graph, reading other data files, or making a Gemini call.
- If it exists, build the graph once and invoke it once with {invoice_path: str(path)}.
- Return the final state from invoke.

Main block

- Prompt exactly: Enter the invoice number:
- Strip surrounding whitespace from the entered number.
- If process_invoice returns {}, print: Invoice <invoice_number> was not found
- Otherwise print the report in state[report].

- Importing main.py must not prompt, process an invoice, build the graph, or call an agent.

MODULE 7 — tests.py

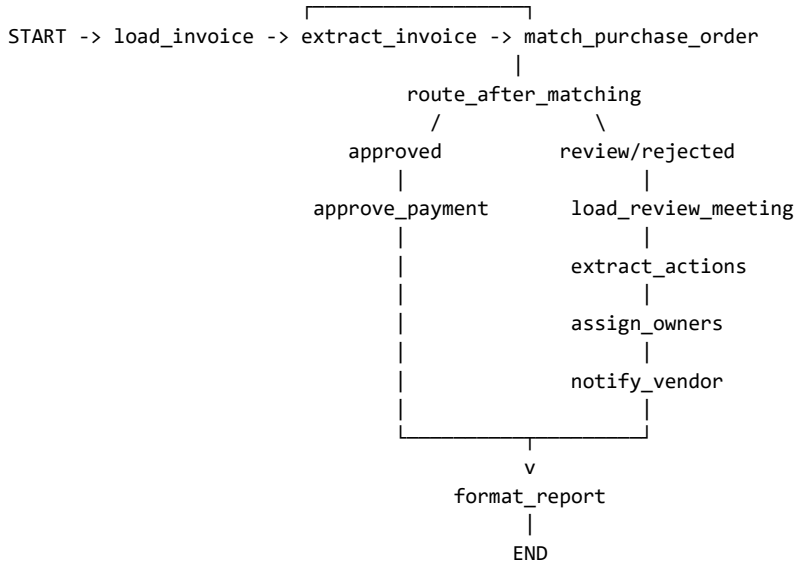
Write pytest tests that isolate Gemini, LangGraph nodes, and tool behavior with mocks or fakes. The normal test run must not require a real API key, make network calls, or mutate supplied datasets. Validate behavior rather than only checking that functions exist. Use temporary files or controlled fixtures where appropriate.

Test area	Minimum assertion
State contract	ProcurementState declares exactly the specified keys; invocation may start with invoice_path only.
Invoice loading	load_invoice reads the complete UTF-8 file and returns only invoice_text.
Structured extraction	InvoiceData rejects missing/wrongly typed fields; extraction agent makes exactly one model call.
Total payable	Extraction handles total payable separately from unit rates, line amounts, and dates.
PO tool	Exact lookup returns all seven columns; approved_amount is float; unknown PO returns {}.
Payment ledger	Duplicate tool returns True for a known invoice and False for an unknown invoice.
Decision precedence	Missing/closed PO takes precedence over duplicate, vendor, and amount mismatches.
Duplicate prevention	Previously paid invoice is rejected and never schedules payment.
Vendor mismatch	Vendor mismatch produces review and a vendor-specific reason.
Amount tolerance	Exactly 2% passes; a difference greater than 2% triggers review.
Deterministic decisions	match_purchase_order makes no Gemini call and applies checks in order.
Payment branch	Only approved invoices call schedule_payment; returned reference is stored unchanged.
Conditional routing	Approved routes to approve_payment; review/rejected route to load_review_meeting.
Meeting lookup	Matching PO copies fields unchanged; missing meeting yields correctly typed empty values.
Action extraction	Only ACTION:-labelled lines are retained; markers removed; order preserved.
Action assignment	One owner/priority call per action; task text and order remain unchanged.
Empty meeting	Empty notes cause no extraction call; empty actions cause no assignment calls.
Vendor note	Review/rejected path makes one validated note call; approved path makes none.
Report formatting	Approved report includes payment reference; non-approved report includes note and actions.
Missing invoice	Returns {} without building graph, reading other data, or calling an agent.
Import safety	Importing main.py does not prompt or process data.
Graph construction	build_graph performs no data reads, Gemini calls, or payment scheduling.
Call-count integration	Approved path: one Gemini call. Non-approved path with actions: one invoice extraction, one action extraction, one call per action, one vendor note; no extra calls for empty notes.
Regression / edge cases	Unknown PO, duplicate invoice, closed PO, missing meeting, no actions, malformed agent output, and invalid schemas are covered.

Minimum coverage Implement at least 24 meaningful tests. Assert tool/agent call counts and branch behavior, not only final text. You may add tests, but must not weaken any listed requirement.

WORKFLOW ARCHITECTURE

The workflow is a directed graph with one conditional split and a shared report node.



AGENT / TOOL RESPONSIBILITY MATRIX

Component	Gemini?	May decide approval?	Allowed responsibility
extract_invoice_data	Yes	No	Extract and validate invoice fields.
extract_follow_up_actions	Yes	No	Extract action statements from meeting prose.
assign_action_owner_priority	Yes	No	Choose one attendee and a priority per task.
draft_vendor_note	Yes	No	Draft a note from an already-determined decision.
lookup_purchase_order	No	No	Read a PO row from CSV.
invoice_already_paid	No	No	Check the read-only payment ledger.
schedule_payment	No	No	Return deterministic simulated payment reference.
match_purchase_order	No	Yes, fixed Python rules only	Apply ordered business checks.
route_after_matching	No	No	Route using the decision field only.

EXPECTED OUTPUT

Exact vendor-note wording and extracted actions may vary because they are model-generated. Report labels, decision, payment-reference format, section ordering, and missing-record handling must follow this specification.

Example A — approved invoice

```
Enter the invoice number: INV-25041

PROCUREFLOW AI — INVOICE REVIEW
Invoice Number : INV-25041
Vendor         : Northstar Components Ltd.
Purchase Order : PO-8801
Decision       : approved
Reason         : All payment eligibility checks passed.
Payment Ref.   : PAY-INV-25041-48500
```

Example B — review invoice with meeting actions

```
Enter the invoice number: INV-25043

PROCUREFLOW AI — INVOICE REVIEW
Invoice Number : INV-25043
Vendor         : Orion Industrial Supplies
Purchase Order : PO-8803
Decision       : review
Reason         : Invoice total exceeds the permitted 2% tolerance.

VENDOR NOTE
Subject : Clarification required for invoice INV-25043
Message : <courteous message returned by the vendor-note agent>

PROCUREMENT FOLLOW-UP
Meeting ID : PRM-204
Meeting    : October PO Exception Review
Actions (2):
  1. [High] Reconcile the tax-inclusive total with the approved PO.
     Owner: Meera Shah
  2. [Medium] Confirm the revised delivery schedule with the supplier.
     Owner: Arjun Rao
```

Example C — invoice file missing

```
Enter the invoice number: INV-25999
Invoice INV-25999 was not found
```

Examples illustrate formatting only. The actual dataset determines invoice details, decisions, payment references, and generated text. Do not hard-code example values.

IMPLEMENTATION REQUIREMENTS

Environment setup

1. Create and activate a virtual environment.
2. Install dependencies using: pip install -r requirements.txt
3. Copy .env.example to .env and set GEMINI_API_KEY and GEMINI_MODEL.
4. Implement the modules in the specified project structure.
5. Run the application from the Project directory: python3 main.py
6. Run tests using: python3 -m pytest tests.py -vv

Required dependencies

Include compatible versions of langgraph, langchain-core, google-genai, pydantic, python-dotenv, pandas, and pytest in requirements.txt. Add another dependency only if required by the implementation.

ACCEPTANCE CRITERIA

- Exactly four LLM agent functions exist, with the names and responsibilities defined here.
- Payment decisions are made entirely by deterministic code using the ordered rules.
- The graph has separate approved and non-approved branches that converge at format_report.
- All tool calls use invoke with argument dictionaries.
- No model call occurs during graph construction, for a missing invoice, or on the approved branch after invoice extraction.
- The owner/priority agent is called once per action, preserving task order.
- Data files remain unchanged after every run.
- Tests run without a real Gemini API call; the application is import-safe.
- No hard-coded invoice decisions, payment references, vendor notes, or meeting actions are used to fake a result.
- The report includes branch-specific sections and handles a missing linked meeting gracefully.

SUBMISSION CHECKLIST

- ☐ All Python modules and requirements.txt are present.
- ☐ .env.example is present; no real API key is submitted.
- ☐ All four agent functions use the required output formats and schemas.
- ☐ The graph compiles and follows the specified topology.
- ☐ At least 24 meaningful pytest tests pass.
- ☐ The data directory and all four dataset groups are present.
- ☐ README explains installation, environment setup, execution, and testing.
- ☐ Console report follows the required structure.